

1999
©MJRL

Estratégias de Teste

- **Uma estratégia de teste deve contemplar:**
  - **Plano de testes**
  - **Projeto dos casos de teste**
  - **Execução dos testes**
  - **Coleta de resultados e avaliação**
- **Uma estratégia de teste deve ser flexível para promover a criatividade e ajustes que venham ser necessários.**
- **Deve ser rígida para permitir o acompanhamento do seu progresso em relação ao que foi planejado.**

1999
©MJRL

Estratégias de Teste

- **Uma estratégia de teste deve acomodar testes de baixo nível necessários para a verificação da implementação de partes do código fonte e também testes de alto nível para a comprovação de atendimento a requisitos.**
- **Deve fornecer referências aos gerentes e um fonte de orientação aos desenvolvedores, através de um processo mensurável.**

1999
©MJRL

Verificação & Validação

- **A atividade de teste de software faz parte de um conjunto mais amplo de atividades denominado Verificação e Validação (V&V).**
- **Verificação refere-se ao conjunto de atividades que garante que o software implementa corretamente um função específica.**

  "Estamos construindo certo o produto?"

- **Validação refere-se ao conjunto de atividades que garante que o software que foi construído é "rastreável" às exigências do cliente.**

  "Estamos construindo o produto certo?"

1999
©MJRL

No programa Apollo, 80 % do esforço total de desenvolvimento ocorreu na fase de testes.

Dunn, 1984

1999
©MJRL

Organização para o Teste de Software

- **Existe um inerente conflito de interesses quando o teste se inicia.**

Quem desenvolve quer demonstrar que seu código não possui erros.

Quem testa quer mostrar que o código possui erros.

1999
©MJRL

Engenharia de Software

Organização para o Teste de Software



- **Do ponto de vista psicológico a análise e o projeto são atividades CONSTRUTIVAS**



- **Já o teste pode ser considerado como uma atividade DESTRUTIVA**

1999
©MJRL

Organização para o Teste de Software

- **Existem alguns pensamentos equivocados sobre os testes:**
  - **Quem desenvolve o software não pode se envolver em nenhuma atividade de teste**
  - ***O desenvolvedor é responsável pelos testes individuais das unidades do código (classes, funções, arquivos, etc) garantindo que estes as funções para as quais foram desenvolvidos.***
    Em alguns casos o desenvolvedor realiza também os testes de integração.

1999
©MJRL

Organização para o Teste de Software

- **O software deve ser testado sem misericórdia por pessoas estranhas ao desenvolvimento**
- ***O papel de um Grupo de Teste Independente (GTI) é encontrar problemas que, dado o envolvimento do programador com o processo de desenvolvimento, não foram encontrados nos testes de unidade.***

1999
©MJRL

Organização para o Teste de Software

- **O GTI deve ser envolvidos no processo somente na etapa de testes.**
- ***GTI faz parte da equipe de projeto de desenvolvimento de software, no sentido de estarem envolvidos durante o processo de especificação e continuam planejando e especificando procedimentos de teste ao longo de um grande projeto.***

1999
©MJRL

Organização para o Teste de Software - Exemplo

- **A Microsoft adota a filosofia de GTI. Segundo alguns autores, ao iniciar um projeto cada desenvolvedor tem designado um responsável por testes.**
- **Os testes iniciam-se em conjunto com a fase de codificação onde são geradas versões diárias, a fim de verificar aspectos de integração. Este tipo de teste é chamado de teste de fumaça (*smoke test*).**

1999
©MJRL

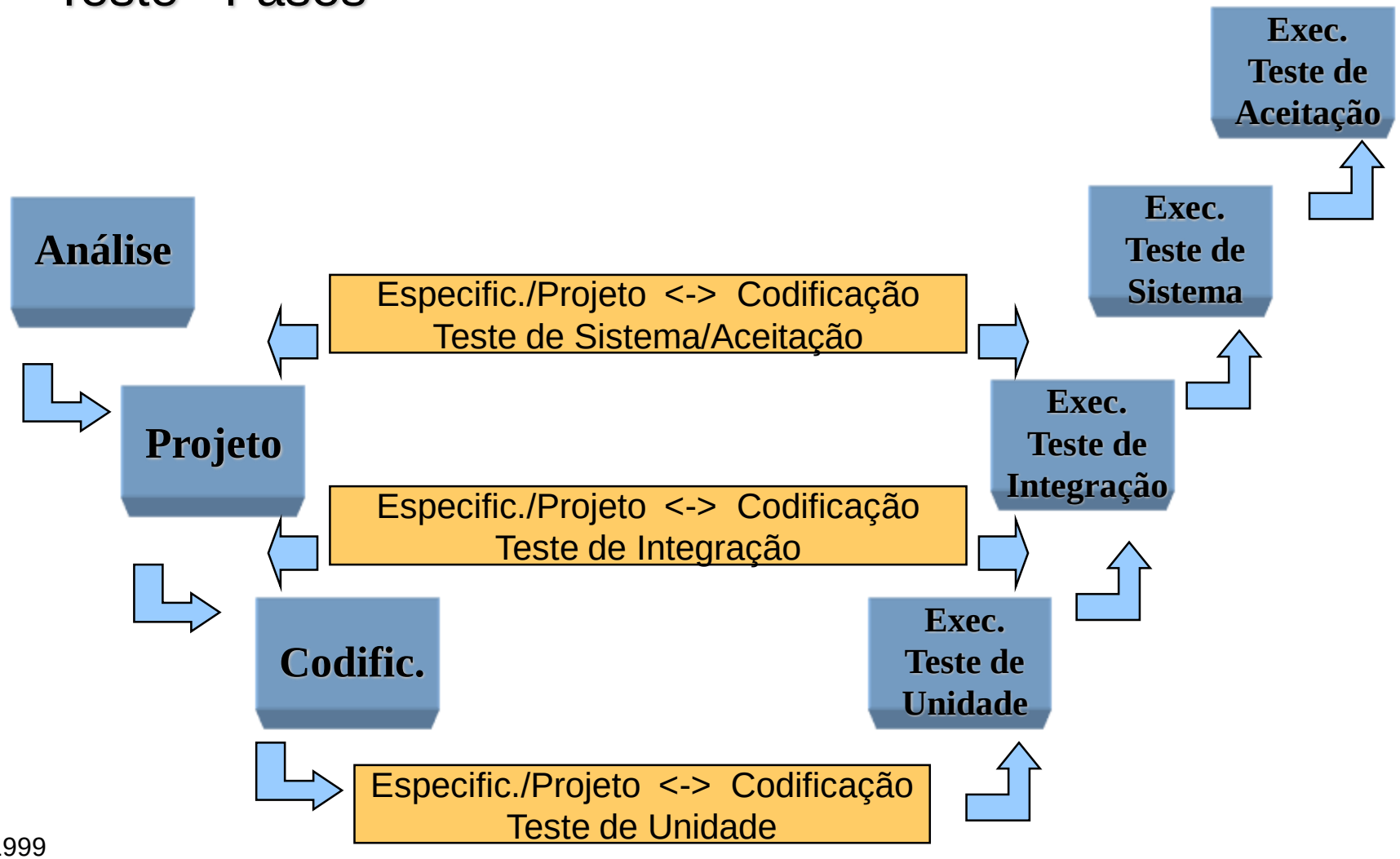
Testes - Fases

- **Na atividade de teste são consideradas geralmente 4 fases:**
  - **Teste de unidade**
  - **Teste de integração**
  - **Teste de sistema**
  - **Teste de aceitação**

1999
©MJRL

Engenharia de Software

Teste - Fases



1999
©MJRL

Teste - Fases

- **Teste de Unidade: cada unidade de software (classe, procedimento, função, arquivo, etc) é testado individualmente garantindo que ele funcione adequadamente.**
- **Testes de Integração: as unidades são montadas ou integradas em módulos e então testadas. Os módulos, por sua vez, são integrados e o software é testado como um todo.**

1999
©MJRL

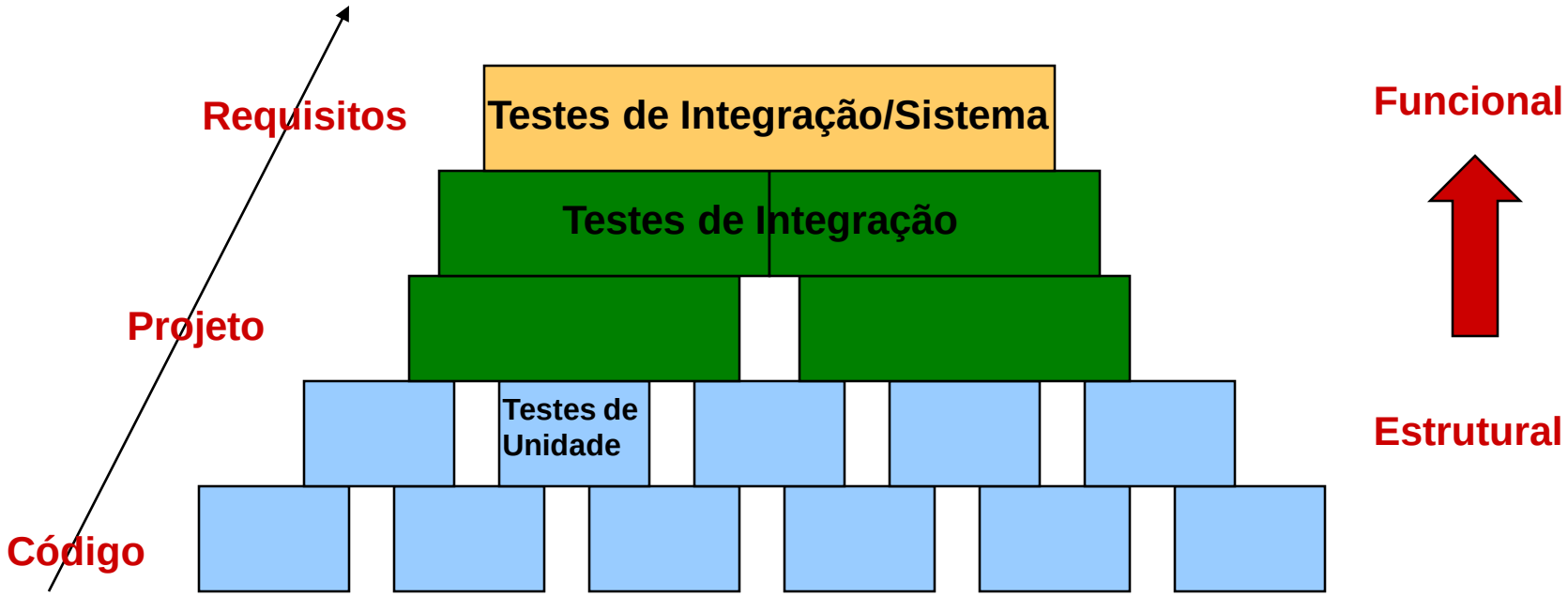
Teste - Fases

- **Teste de Sistema: todos os módulos de software são integrados e o software testado em conjunto com o hardware.**
- **Testes de Aceitação: são testes realizados com a presença do cliente. Geralmente são repetidos alguns dos testes de sistema com o objetivo de demonstrar suas funcionalidades.**

1999
©MJRL

Engenharia de Software

Teste - Fases



1999
©MJRL

Teste de Unidade

- **A *interface* com o módulo é testada para ter a garantia de que as informações fluem para dentro e para fora da unidade de programa que se encontra sob teste**
- **A *estrutura de dados* local é examinada para ter a garantia de que dados armazenados temporariamente mantêm sua integridade durante todos os passos de execução.**

1999
©MJRL

Teste de Unidade

- **As *condições de limite* são testadas para ter a garantia de que o módulo opera adequadamente nos limites estabelecidos para demarcarem ou restringirem o processamento.**
- **Todos os *caminhos independentes* através da estrutura de controle são exercitados para ter a garantia de que todas as instruções de um módulo foram executadas pelo menos uma vez.**

1999
©MJRL

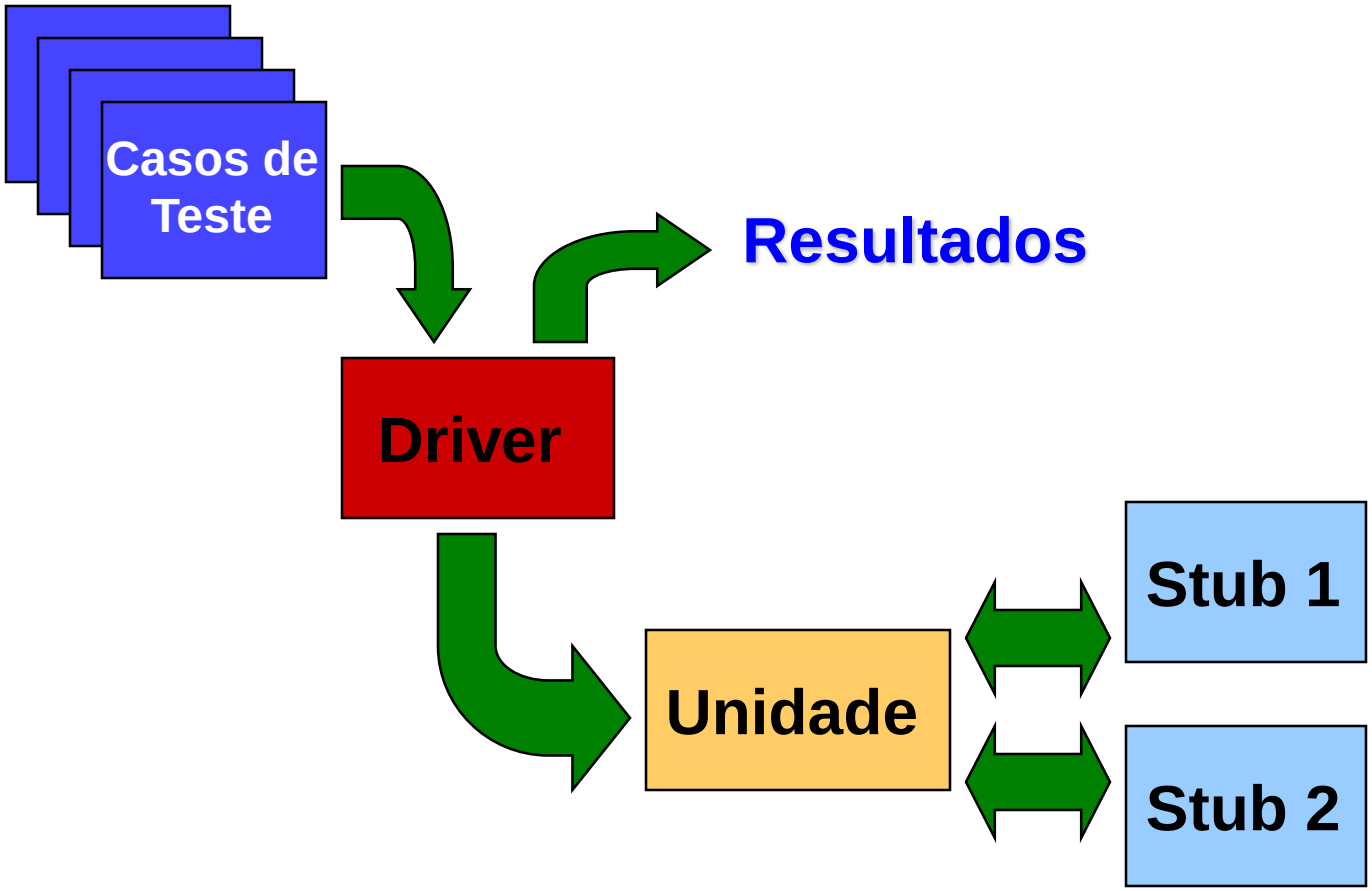
Teste de Unidade

- **Cada unidade deve ser testada isoladamente, assim dois elementos se fazem necessários:**
  - ***Driver*: Programa que basicamente invoca a unidade em teste, passando parâmetros e recebendo resultados.**
  - ***Stub*: Substitui outras unidades que porventura sejam invocadas pela unidade em teste.**

1999
©MJRL

Engenharia de Software

Teste de Unidade



1999
©MJRL

Engenharia de Software

Teste de Unidade

```
Procedimento MAIOR(A:VETOR;
T:inteiro; var MAX:inteiro);

variáveis  I,M:inteiro
Inicio
   M<- A[1];
   I<-2;
   enquanto I < T faça
       se A[I] > M
         então M<- A[I]
               I<- I+1
         senão I<- I+1
       fim se
   fim enquanto
   MAX<-M;
   fim do procedimento
```

- **Considere a unidade de software ao lado.**

1999
©MJRL

Teste de Unidade

- **Casos de teste funcionais**

Caso de Teste	Entradas	Resultados Esperados	Tipo do Teste
T-01	A[] = 1,2,3,4 T = 4	MAX = 4	Funcional
T-02	A[] = 4,3,2,1 T = 4	MAX = 4	Funcional
T-03	A[] = 2 T = 1	MAX = 2	Funcional
T-04	A[] = T = 0	ERRO	Funcional
T-05	A[] = 3,3,3,3 T = 4	MAX = 3	Funcional

1999
©MJRL

Teste de Unidade

- **Casos de teste estruturais**

Caso de Teste	Entradas	Resultados Esperados	Tipo do Teste	Caminho Percorrido
T-06	A[] = 3,4 T = 2	MAX = 4	Estrutural	1, 2, 3, 4, 7, 8, 3, 9, 10
T-07	A[] = 4,3 T = 2	MAX = 4	Estrutural	1, 2, 3, 4, 5, 6, 8, 3, 9, 10
T-08	A[] = 2 T = 1	MAX = 2	Estrutural	1,2,3,9,10

1999
©MJRL

Teste de Unidade

- **Driver para o caso de teste T-01**

```
Programa Driver_T-01;
Inicio
  A[1]<- 1;
  A[2]<- 2;
  A[3]<- 3;
  A[4]<- 4;
  T <- 4;
  MAX <- 0;
  MAIOR(A;T;MAX);

  Imprime("Resultado esperado = 4 ");
  Imprime("Maior = ", MAX);
Fim
```

1999
©MJRL

Teste de Integração

- **Técnica sistemática para a construção da estrutura de programa, realizando-se, ao mesmo tempo, testes para descobrir erros associados a interfaces.**
- **O objetivo é montar a estrutura do programa a partir das unidades, tal como foi especificado no projeto (design)**

1999
©MJRL

Teste de Integração

- Integração não incremental: o programa “integrado” por completo e testado como um todo. Quando erros são encontrados, a correção é difícil porque o isolamento das causas é complicado.
- Integração incremental: o programa é construído e testado em pequenos segmentos, onde os erros são mais fáceis de serem isolados e corrigidos; as interfaces têm maior probabilidade de serem testadas completamente e uma abordagem sistemática ao teste pode ser aplicada.

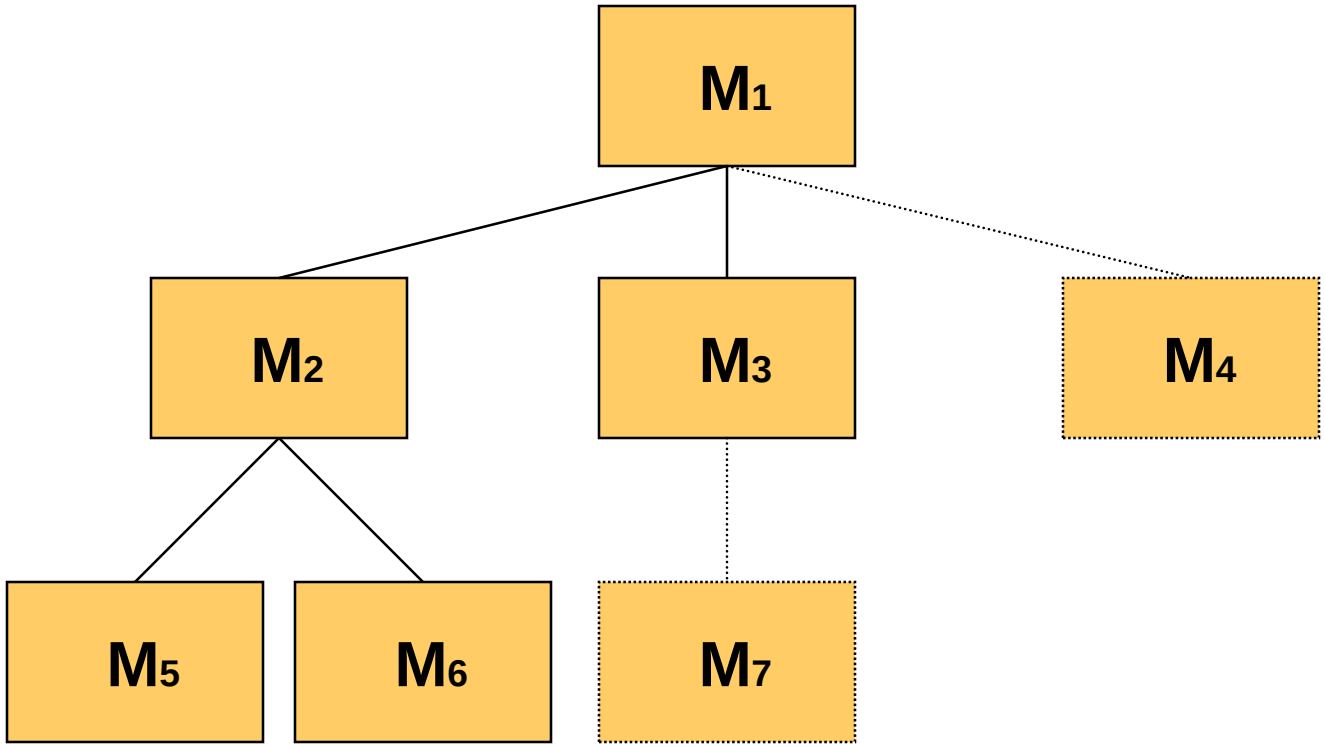
1999
©MJRL

Teste de Integração

- **Os módulos são integrados movimentando-se de cima para baixo, através da hierarquia de controle, iniciando-se do módulo de controle principal (**programa principal**).**
- **Os módulos subordinados ao módulo de controle principal são incorporados à estrutura ou por *depth-first* (**primeiro pela profundidade**) ou *breadth-first* (**primeiro pela largura**).**

1999
©MJRL

Teste de Integração



Depth-First: M_1, M_2, M_5
Breadth-First: M_2, M_3, M_4

1999
©MJRL

Teste de Integração - Top-Down

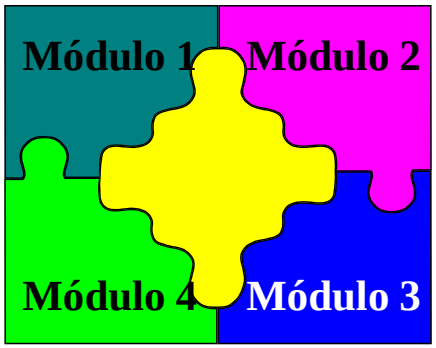
- **Passos do processo de integração:**
  - O módulo de controle é usado como um *driver* de teste e os *stubs* são substituídos para todos os módulos diretamente subordinados ao módulo de controle principal.
  - Dependendo da abordagem de integração escolhida, os *stubs* subordinados são substituídos, um de cada vez por módulos reais.
  - Testes são realizados à medida que cada módulo é integrado.

1999
©MJRL

Teste de Integração - Top-Down

- Durante a conclusão de cada conjunto de testes, outro *stub* é substituído pelo módulo real.
- *Teste de regressão* - (realização de todos ou de alguns dos testes anteriores) pode ser realizado a fim de garantir que novos erros não tenham sido introduzidos.





1999
©MJRL

Teste de Integração - Bottom-Up

- **Passos do processo de integração:**
  - Os módulos de baixo nível são combinados em clusters que executam uma única função específica.
  - Um driver é escrito para coordenar as entradas e saídas do caso de teste.
  - O cluster é testado
  - Os drivers são removidos e os clusters são combinados movendo a integração de baixo para cima.

1999
©MJRL

Teste de Integração - Teste de Validação

- Ao término da atividade de teste de integração, o software está completamente montado como um pacote, erros de interface foram descobertos e corrigidos e uma série final de testes de software - os *testes de validação* - pode iniciar-se.
- A validação é bem sucedida quando o software funciona de uma maneira razoavelmente esperada pelo cliente.

1999
©MJRL

Teste de Integração - Teste de Validação

- **A *Especificação de Requisitos de Software* contém os *critérios de validação* que formam a base para o teste de validação.**
- **A validação do software é realizada por meio de uma série de testes de caixa preta que demonstram a conformidade com os requisitos.**

1999
©MJRL

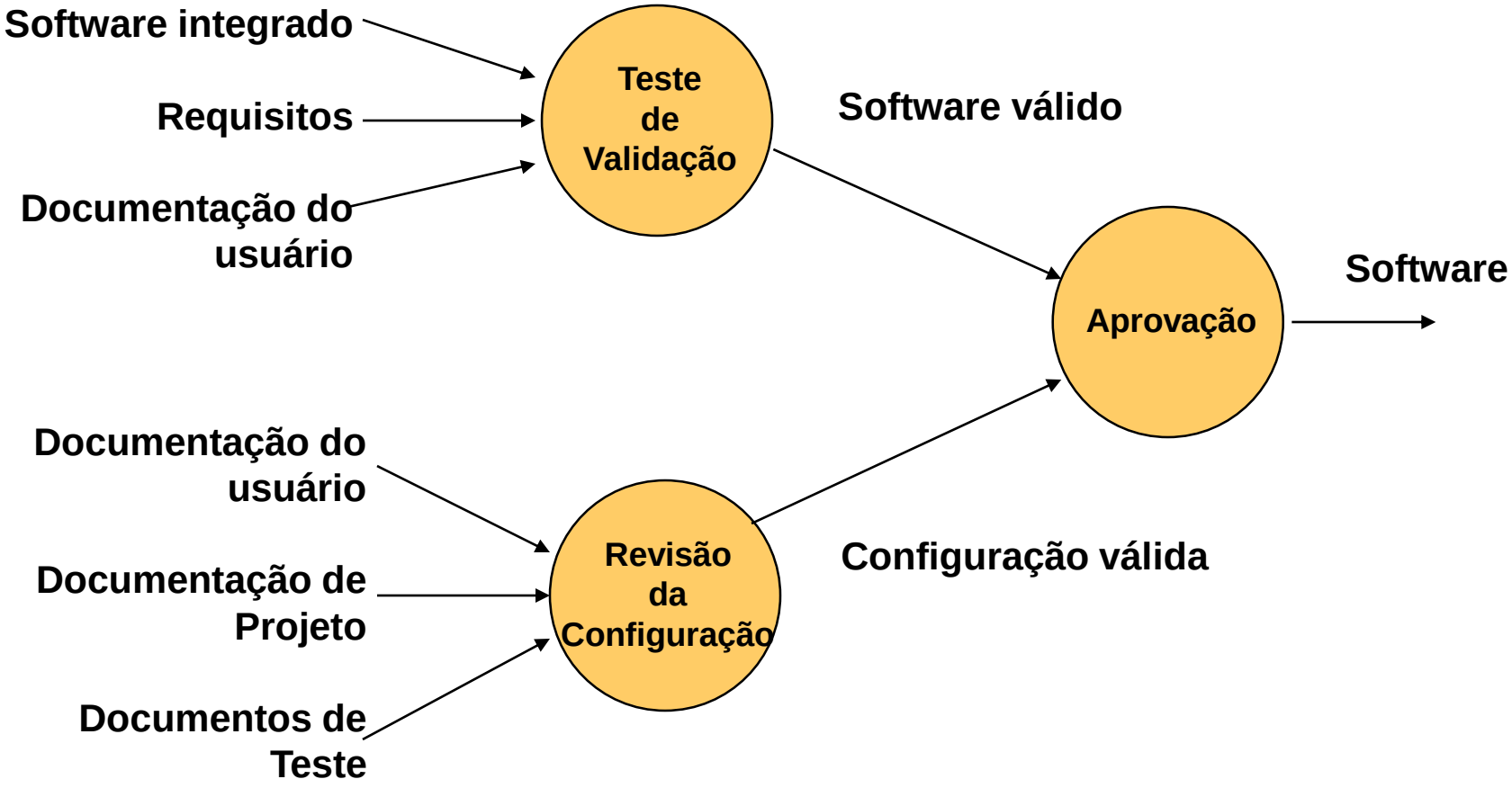
Teste de Integração - Teste de Validação

- Um elemento importante do processo de validação é a *revisão de configuração*.
- O propósito dessa revisão é garantir que todos os elementos da configuração de software tenham sido desenvolvidos de maneira adequada, estão catalogados e têm os detalhes necessários para apoiar a fase de manutenção do ciclo de vida do software.
- Algumas vezes essa atividade é chamada *Auditoria*.

1999
©MJRL

Engenharia de Software

Teste de Integração - Teste de Validação



1999
©MJRL

Teste de Integração - Testes Alfa e Beta

- **É virtualmente impossível que se preveja como o cliente realmente usará um programa.**
- **As instruções de uso podem ser mal-interpretadas, combinações estranhas de dados podem ser regularmente usadas, saídas que pareciam claras ao analista podem ser ininteligíveis para um usuário.**
- **Duas estratégias são freqüentemente utilizadas: Testes Alfa e Beta.**

1999
©MJRL

Teste de Integração - Testes Alfa e Beta

- ***Teste Alfa*** - é realizado por um cliente nas instalações do desenvolvedor. O software é usado num ambiente controlado com o desenvolvedor "olhando sobre os ombros" do usuário e registrando erros e problemas de uso.
- ***Teste Beta*** - é realizado nas instalações do cliente pelo usuário final do software. O desenvolvedor não está presente. O cliente registra os problemas que são encontrados e relata-os ao desenvolvedor a intervalos regulares.

1999
©MJRL

Teste de Integração - Teste de Sistema

- **O *teste de sistema* envolve uma série de diferentes testes, cujo propósito primordial é exercitar integralmente o sistema como um todo.**
- **São utilizados testes tais como: recuperação de falhas, segurança, stress e performance.**

1999
©MJRL

Teste de Integração - Teste de Sistema

- **Teste de recuperação**
  - Muitos sistemas baseados em computador precisam recuperar-se de falhas e retomar o processamento dentro de um tempo previamente especificado.
  - Em certos casos, o sistema deve ser tolerante a falhas, isto é, o processamento de falhas não deve fazer com que uma função global de sistema seja interrompida.
  - O teste de recuperação é um teste de sistema que força o software a falhar de diversas maneiras e verifica se a recuperação é adequadamente executada.

1999
©MJRL

Teste de Integração - Teste de Sistema

- **Teste de Segurança**
  - Qualquer sistema baseado em computador que gerencie informações sensíveis ou provoque ações que possam impropriamente prejudicar (ou beneficiar) pessoas constitui um alvo para o acesso impróprio ou ilegal.
  - O teste de segurança tem por objetivo verificar se todos os mecanismos de proteção embutidos num sistema o protegerão de acessos indevidos.

1999
©MJRL

Teste de Integração - Teste de Sistema

- Durante o teste de segurança, o testador desempenha o papel de pessoa que deseja invadir no sistema. Os recursos usados são:
  - Adquirir senhas mediante contatos externos
  - Atacar o sistema com software customizado projetado para derrubar quaisquer defesas que tenham sido construídas
  - Desarmar o sistema, negando serviço a outros
  - Provocar erros intencionalmente, esperando acessá-lo durante a recuperação
  - ”Paginar” através de dados inseguros esperando descobrir a chave de entrada no sistema.

1999
©MJRL

Teste de Integração - Teste de Sistema

- **Teste de Stress**
  - Esse tipo de teste é projeto para confrontar o sistema com situações de operação anormais.
  - Testes de stress executam um sistema de maneira que os seus recursos sejam submetidos a quantidades, freqüências ou volumes anormais.
  - Em síntese, o testador tentar esgotar o sistema.

1999
©MJRL

Teste de Integração - Teste de Sistema

- **Teste de Performance**
  - São projetados para testar a performance em tempo de execução de um software no contexto de integração do sistema.
  - Ocorrem em todos os níveis de teste (p.ex. unidade e integração)
  - Porém uma visão completa da performance do sistema somente será obtida após a sua completa integração.

1999
©MJRL

Depuração

- **O objetivo da depuração é descobrir e corrigir a causa de um erro de software.**

- **A depuração ocorre em conseqüência de testes bem sucedidos. Embora a depuração possa e deva ser ser um processo disciplinado, ela ainda tem muito de arte.**

1999
©MJRL

Depuração

- **Por que depurar é tão difícil?**
  - O sintoma e a causa podem ser geograficamente remotos. Estruturas altamente acopladas exarcebam essa situação.
  - O sintoma pode desaparecer (temporariamente) quando outro erro é corrigido.
  - O sintoma pode, de fato, ser causado por não erros (por exemplo, imprecisões de arredondamento).
  - O sintoma pode ser causado por um erro humano que não é facilmente rastreado.
  - O sintoma pode ser resultado de problemas de *timing*, e não problemas de processamento.

1999
©MJRL

Depuração

- **Força Bruta**
  - É provavelmente é o método mais comum e menos eficiente de isolar a causa de um erro de software. É aplicado quando tudo o mais falha.
  - Usa-se uma filosofia do tipo "deixe que o computador descubra o erro". Por exemplo, são feitas listagens de memória (*memory dumps*), são inseridas instruções WRITE, etc.
  - Não obstante, a massa de informações produzida possa levar ao sucesso, mais freqüentemente ela conduz a tempo e esforço desperdiçados.

1999
©MJRL

Depuração

- **Backtracking**
  - É uma abordagem que pode ser usada com sucesso em pequenos programas.
  - Iniciando-se no local em que o sintoma foi descoberto, o código fonte é rastreado para trás (manualmente) até que o local da causa seja encontrado.
  - Infelizmente, à medida que o número de linhas de código aumenta, o número de potenciais caminhos de *backtracking* pode tornar-se extremamente alto.

1999
©MJRL

Depuração

- **Eliminação da Causa**
  - Os dados relacionados à ocorrência de erros são organizados para isolar potenciais causas.
  - Uma "hipótese de causa" é imaginada e os dados são usados para provar ou refutar a hipótese.
  - Alternativamente, uma lista de todas as causas possíveis é desenvolvida e testes são realizados para eliminar cada uma.
  - Se os testes iniciais indicarem que uma hipótese de causa apresenta possibilidades, os dados são refinados, numa tentativa de isolar o *bug*.

1999
©MJRL